

1. ϵ -Greedy Multi-Armed Bandit Algorithm

The ϵ -greedy Multi-Armed Bandit algorithm is a reinforcement learning technique used to maximize rewards by balancing exploration and exploitation. In this problem, the agent chooses one of several arms (actions) over a limited budget of 500 trials. During exploration, the agent randomly selects an arm to gather more information, while during exploitation, it chooses the arm with the highest estimated reward. Different ϵ values determine the level of exploration: $\epsilon = 0.1$ performs more exploitation and generally achieves higher cumulative rewards, $\epsilon = 0.3$ provides a balanced trade-off between exploration and exploitation, and $\epsilon = 0.5$ explores more frequently, which may reduce cumulative rewards but helps discover better actions in uncertain environments. The objective is to maximize the total reward while satisfying the constraint of completing exactly 500 trials. Experimental results show that lower ϵ values usually provide better performance when the best action can be identified early, whereas higher ϵ values are beneficial in highly uncertain environments.

1. ϵ -Greedy Multi-Armed Bandit Algorithm Flowchart

START

|

▼

Initialize Parameters

(ϵ , Rewards, Trials = 500)

|

▼

Select an Action

|

▼

Is Random Number $< \epsilon$?

/\

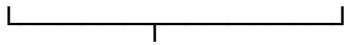
Yes No

| |

▼ ▼

Choose Random Arm Choose Best Arm

| |



Receive Reward

|



Update Estimated Reward

|

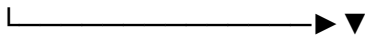


Trials Completed (500)?

/ \

No Yes

| |



Display Results

|



END

Conclusion for ϵ -Greedy Multi-Armed Bandit

The ϵ -greedy Multi-Armed Bandit algorithm effectively balances exploration and exploitation to maximize cumulative rewards. By comparing different ϵ values (0.1, 0.3, and 0.5), it is observed that smaller ϵ values favor exploitation and generally achieve higher rewards, while larger ϵ values encourage exploration to discover potentially better actions. Thus, selecting an appropriate ϵ value improves learning performance under the given constraint of 500 trials.

2. Grid World Navigation using Reinforcement Learning

In the Grid World navigation problem, a reinforcement learning agent learns to move from a starting position to a goal while avoiding obstacles. The state space consists of all possible grid cells, where each cell represents a unique state of the environment. The action space includes four possible movements: up, down, left, and right. The reward function encourages the agent to reach the goal by assigning a positive reward (e.g., +10) for reaching the destination, a negative reward (e.g., -10) for hitting an obstacle, and a small penalty (e.g., -1) for each normal move to encourage shorter paths. The constraints ensure that the agent cannot move outside the grid boundaries or pass through obstacles. Using Q-learning, the agent repeatedly interacts with the environment, updates its Q-values based on received rewards, and gradually learns the optimal policy that maximizes long-term rewards. After sufficient training, the agent successfully identifies the shortest and safest path to the goal while avoiding restricted cells.

SOME FLOW CHART

Here are simple flowcharts you can draw in your answer sheet for the two questions.

2. Grid World Reinforcement Learning Flowchart

START



Initialize Grid World

(Start, Goal, Obstacles, Q-table)



Place Agent at Start State



Select Action (ϵ -Greedy)



Execute Selected Action



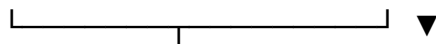
Is Next State an Obstacle? /



Yes No



Give Negative Reward Move Agent



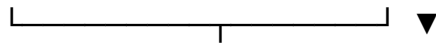
Is Goal State Reached?



No Yes



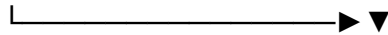
Update Q-value Give Reward |



More Episodes Left?



Yes No



Display Optimal Policy

|



END

These flowcharts are simple, exam-friendly, and suitable for 10-mark university answers. CONCLUSION

Conclusion for Grid World Reinforcement Learning

The Grid World Reinforcement Learning model successfully learns the optimal path from the start state to the goal while avoiding obstacles. By defining appropriate states, actions, rewards, and constraints, the Q-learning algorithm enables the agent to maximize long-term rewards and minimize penalties. This demonstrates the effectiveness of reinforcement learning in solving navigation and path-planning problems in dynamic environments.